Rapport de Projet – Developpement Web ING1

Plateforme Maison Intelligente (Spring Boot + React)

Khayem Ben Ghorbel Rindra Ziyad Anastasia Valerie

Annee universitaire 2025–2026

Table des matières

1 Introduction **2**

2 Objectifs et perimetre **2**
2.1 Objectifs fonctionnels 2
2.2 Objectifs non-fonctionnels 2

3 Architecture technique **2**
3.1 Stack 2
3.2 Organisation du code 2
3.3 Routes REST principales 2

4 Methodologie projet **3**
4.1 Repartition des contributions 3

5 Couverture fonctionnelle **3**
5.1 Module Information 3
5.2 Module Visualisation 3
5.3 Module Gestion 3
5.4 Module Administration 3

6 Securite et controle d'acces **4**

7 Schema UML complet (grande vue) **4**

8 Schema de base de donnees complet (grande vue relationnelle) **6**
8.1 Choix de modelisation et performances 8

9 Tests et validation **8**
9.1 Builds 8
9.2 Tests de parcours demo 8
9.3 Responsive et UX 8

10 Deploiement **8**

11 Difficultes rencontrees et solutions **8**

12 Conclusion **8**

1 Introduction

Ce document presente la realisation du projet de Developpement Web ING1 autour du theme **Maison Intelligente**. L'objectif est de livrer une application web complete, avec architecture full-stack, base de donnees persistante, gestion des roles, automatisation domotique et module d'administration.

Le projet respecte les attentes du module :

— frameworks backend/frontend,
— persistance des donnees,
— recherche multi-criteres,
— separation claire des modules,
— demonstration fonctionnelle complete.

2 Objectifs et perimetre

2.1 Objectifs fonctionnels

1. **Information** : consultation publique des pieces, objets et services.
2. **Visualisation** : espace membre (authentification, profil, consultation detaillee).
3. **Gestion** : pilotage avance des objets, scenarios, maintenance, statistiques.
4. **Administration** : gestion utilisateurs, droits et validation des comptes.

2.2 Objectifs non-fonctionnels

— interface responsive,
— securite d'acces par role et niveau,
— traces d'actions,
— disponibilite locale et deploiement cloud (Render).

3 Architecture technique

3.1 Stack

— **Backend** : Java 21, Spring Boot 3.3, Spring Data JPA, Validation, Mail SMTP.
— **Frontend** : React 19, Vite.
— **BDD** : H2 (fichier local) et base MySQL prete.
— **Deploiement** : Render (API Docker + Frontend statique).

3.2 Organisation du code

— `backend/` : modeles, DTO, repositories, services, controllers.
— `frontend/` : pages, composants, etat UI, appels REST.
— racine : `pom.xml`, `Dockerfile`, `render.yaml`, `README.md`.

3.3 Routes REST principales

— `/api/info/*`
— `/api/auth/*`
— `/api/visualisation/*`
— `/api/gestion/*`
— `/api/admin/*`

4 Methodologie projet

Le travail a ete mene de maniere iterative (inspire SCRUM) :

— priorisation du backlog et progression par increments,
— cycles courts implementation + verification,
— demonstrations regulieres des fonctions importantes,
— stabilisation finale pour soutenance.

4.1 Repartition des contributions

Membre	Role principal	Contributions
Khayem Ben Ghorbel	Pilotage technique	Architecture globale, integration front/back, securite auth, stabilisation demo
Rindra	Implementation fonctionnelle	Support modules Information/Visualisation, validations de parcours
Ziyad	Support backend	Verifications API, aides sur robustesse modele/controleurs
Anastasia	UX / presentation	Qualite visuelle des ecrans, coherence navigation et lisibilite
Valerie	Documentation / soutenance	Structuration rapport, support scenario de demonstration

5 Couverture fonctionnelle

5.1 Module Information

— affichage des pieces et objets,
— recherche avec filtres combines,
— acces sans connexion.

5.2 Module Visualisation

— inscription / connexion / profil,
— consultation des objets et des services,
— progression utilisateur (points + niveau).

5.3 Module Gestion

— CRUD objets connectes,
— commandes specialisees par type (machine cafe, aspirateur, etc.),
— scenarios domotiques,
— maintenance priorisee (CRITICAL/HIGH/MEDIUM),
— historique et statistiques.

5.4 Module Administration

— gestion des utilisateurs,
— attribution/retrait du role admin,
— validation/suspension des comptes,
— traitement des demandes de suppression.

6 Securite et controle d'acces

Le flux de securite applique est le suivant :

1. inscription,
2. verification email par code (envoi SMTP reel),
3. validation manuelle par administrateur,
4. connexion autorisee uniquement apres ces etapes.

Un compte non approuve est bloque au login (retour HTTP 403). Cette approche reduit les inscriptions sauvages et donne un controle operateur sur les nouveaux comptes.

Les acces aux modules sont verifies cote backend :

— module Gestion reserve aux utilisateurs de niveau `AVANCE`,
— module Administration reserve aux comptes `admin=true`.

7 Schema UML complet (grande vue)

Le schema UML ci-dessous presente l'ensemble des entites metier, leurs heritages, leurs compositions et les liens majeurs entre modules.

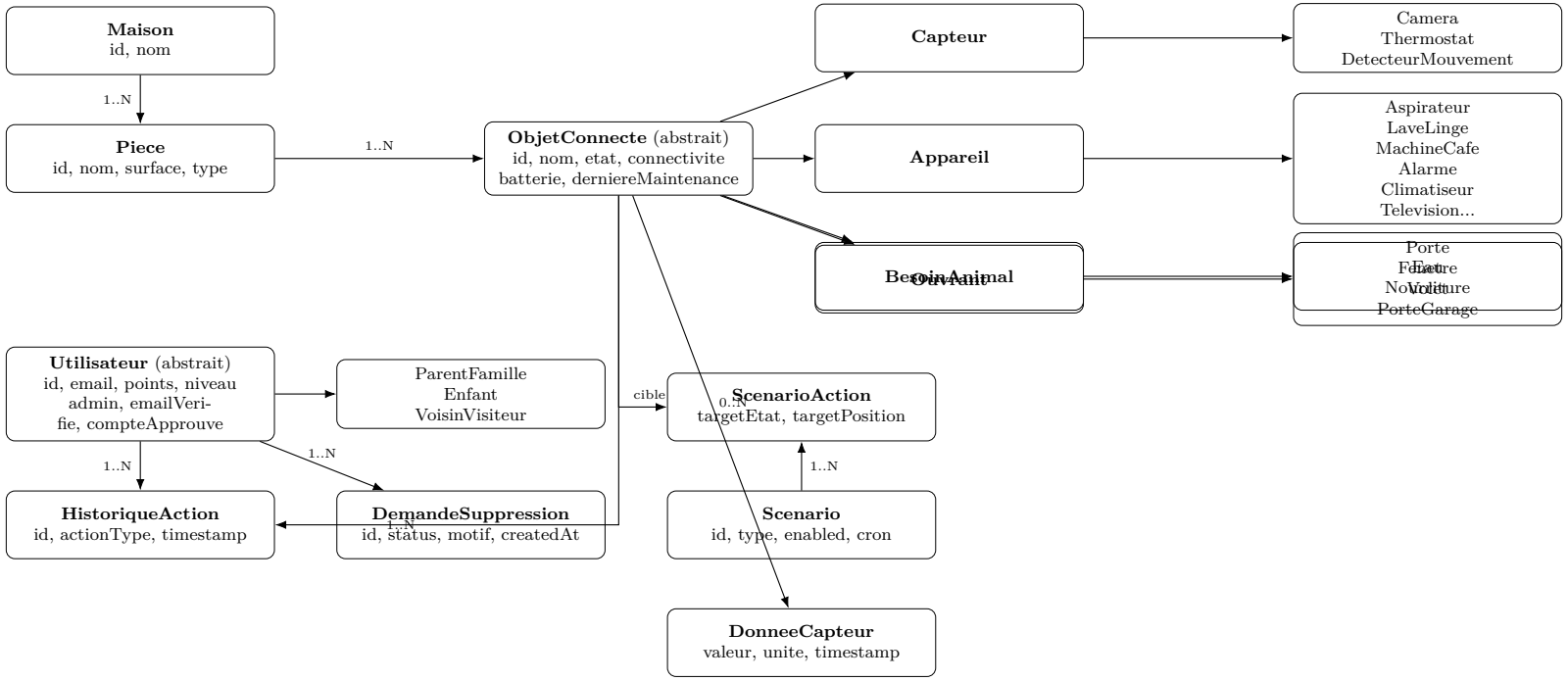

FIGURE 1 – Schema UML global (vue complete et agrandie).

8 Schema de base de donnees complet (grande vue relationnelle)

La base de donnees est organisee autour des tables coeur `utilisateur`, `objet_connecte`, `piece`, et des tables de liaison/fonctionnement (historique, scenarios, demandes, donnees capteurs).

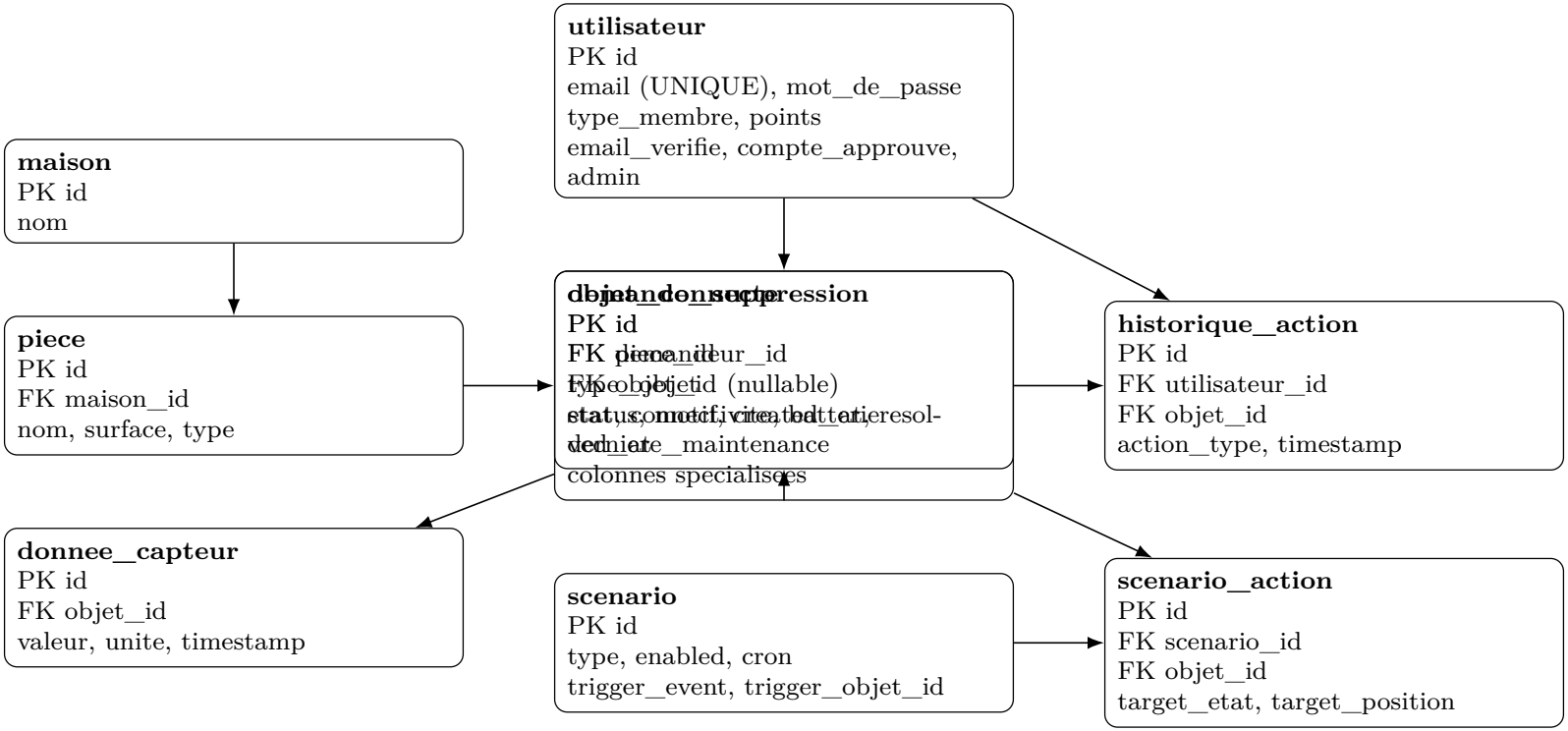

Figure 2 – Schema relationnel principal (PK/FK) de la base.

8.1 Choix de modelisation et performances

- Heritage JPA en `SINGLE_TABLE` pour `utilisateur` et `objet_connecte`.
- Integrite referentielle via FK sur les tables de traces et d'automatisation.
- Requetes cibles a indexer :
  - `objet_connecte(type_objet, piece_id, etat)`
  - `historique_action(utilisateur_id, objet_id, timestamp)`
  - `scenario(enabled, type, trigger_event)`

9 Tests et validation

9.1 Builds

- Backend : `mvn -DskipTests compile` -> succes.
- Frontend : `npm.cmd run build` -> succes.

9.2 Tests de parcours demo

1. verification endpoint `/api/health`,
2. inscription utilisateur,
3. verification email par code,
4. blocage login tant que validation admin absente,
5. validation admin puis connexion autorisee,
6. execution scenario et actions de maintenance.

9.3 Responsive et UX

- rendu teste en desktop/tablette/mobile,
- feedback utilisateur par toasts et etats explicites,
- sections sensibles masquee/verrouillees selon droits.

10 Deploiement

- API Render : service Docker, healthcheck `/api/health`
- Frontend Render : service statique Vite
- URL racine API (`/`) en 404 : comportement normal d'une API-only

11 Difficultes rencontrees et solutions

1. **Evolution du schema local H2** : gestion des colonnes legacy et compatibilite lors des evolutions modeles.
2. **Synchronisation etat front/back** : mise a jour locale immediate de l'UI apres reponse API pour eviter la latence percue.
3. **Securisation des inscriptions** : ajout verification email reelle + validation admin obligatoire avant login.

12 Conclusion

Le projet livre une plateforme complete, structuree et demonstrable, avec separation claire des modules, modelisation metier riche, automatisation domotique, et securite renforcee.

Les axes d'amelioration prioritaires pour une suite industrielle sont :

- migration full MySQL + scripts de migration formels,
- tests automatises E2E,

— observabilite (metriques, logs structures, alerting).

Annexe – Commandes utiles

```
# Backend
mvn spring-boot:run

# Frontend
cd frontend
npm.cmd run dev

# Build
mvn -DskipTests compile
cd frontend && npm.cmd run build
```